The Decentralization Dilemma: Performance Trade-Offs in IPFS and Breakpoints

Ruizhe Shi
George Mason University
Fairfax, VA, USA
rshi3@gmu.edu

Yuqi Fu
University of Virginia
Charlottesville, VA, USA
jwx3px@virginia.edu

Ruizhi Cheng
George Mason University
Fairfax, VA, USA
rcheng4@gmu.edu

Bo Han
George Mason University
Fairfax, VA, USA
bohan@gmu.edu

Yue Cheng
University of Virginia
Charlottesville, VA, USA
mrz7dp@virginia.edu

Songqing Chen
George Mason University
Fairfax, VA, USA
sqchen@gmu.edu

Abstract

Web 3.0 is redefining the current Web (Web 2.0) with a focus on data and governance decentralization. The InterPlanetary File System (IPFS) exemplifies this shift. However, it faces a trade-off between decentralization and performance: prior studies have shown IPFS's performance degradations but fail to diagnose root causes or deliver actionable fixes.

In this study, we present a comprehensive client-side measurement of IPFS performance, identify key bottlenecks, and propose practical enhancements. The key insights of our study are threefold. First, IPFS suffers from significantly lower (~ 8×) throughput than HTTP, primarily due to its sequential downloading pattern imposed by its default block exchange protocol, Bitswap. By enhancing Bitswap, we manage to increase IPFS's downloading throughput by up to 2.23× for large files. Second, Bitswap's sequential downloading pattern limits parallel data retrieval from multiple sources and prevents BBR congestion control from fully leveraging its throughput optimization potential in lossy networks. Third, Bitswap supersedes DHT as IPFS's primary content discovery tool, resolving >80% of requests. Meanwhile, the reliability of DHT is hampered by unresponsive peers, causing delays or failures. Fine-tuning the activation threshold of DHT could optimize lookup efficiency as we demonstrate.

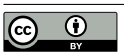
CCS Concepts

• **Networks** → **Network protocol design**; **Application layer protocols**; **Network performance analysis**.

Keywords

IPFS; Network Measurement; Performance Optimization

ACM Reference Format:
Ruizhe Shi, Yuqi Fu, Ruizhi Cheng, Bo Han, Yue Cheng, and Songqing Chen. 2025. The Decentralization Dilemma: Performance Trade-Offs in IPFS and Breakpoints . In *Proceedings of the 2025 ACM Internet Measurement Conference (IMC '25), October 28–31, 2025, Madison, WI, USA.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3730567.3764453



IMC '25, Madison, WI, USA

ACM ISBN 979-8-4007-1860-1/2025/10
https://doi.org/10.1145/3730567.3764453

1 Introduction

Web 2.0, built on centralized client-server architectures [30], has enabled unprecedented connectivity but at the cost of user autonomy and data ownership: today, 90% of global web traffic flows through infrastructure owned by fewer than 10 corporations [18]. This concentration of power has led to systemic vulnerabilities such as censorship, data breaches, and algorithmic manipulation [55], which undermines the Internet's original vision of an open, democratized network. Web 3.0 emerges as a response, promising to redistribute control by replacing centralized intermediaries with decentralized protocols powered by blockchain [14, 38, 47] and peer-to-peer (P2P) technologies. At its core, this movement seeks to reshape the Internet along content-centric principles [72]: data is addressed by what it is (via cryptographic hashes) rather than where it is stored (via URLs or IP addresses), with distributed hash tables (DHTs) acting as the discovery layer for this new paradigm.

Yet, this architectural shift raises fundamental concerns: Can such decentralized systems deliver comparable performance to the traditional centralized systems (*i.e.,* Web 2.0)? This is crucial because users are less likely to switch if they cannot provide a better user experience than the current Web. Content-based addressing and DHTs, while ideologically aligned with Web 3.0's principles, introduce performance trade-offs that are often overlooked. Unlike location-based systems (*e.g.,* HTTP and BitTorrent), which optimize for efficiency by mapping content to predictable servers or swarms, content-centric designs prioritize decentralization, where any node can store and serve any piece of content. These decentralization gains come at the expense of fragmented discovery and delayed retrieval. For instance, resolving a content identifier (CID) requires iterative DHT queries across dozens of nodes, adding seconds of latency before a single byte is transferred [62]. Retrieval then proceeds piece by piece, potentially delayed by unreliable senders.

At the heart of this doubt is IPFS (Interplanetary File System), the de facto storage layer of Web 3.0. IPFS adopts a content-based addressing scheme, mapping data to cryptographic hashes (*i.e.,* CIDs) and relying on the DHT for content discovery (lookup). Since its debut in 2015 [11], IPFS has become increasingly popular, with over 250K active daily nodes, millions of daily end users [53], spanning 152 countries and generating over 10 TB of weekly traffic, according to a recent study [62]. Despite its surging popularity and potential, IPFS fails to deliver comparable performance to Web 2.0 [56, 58]. These shortcomings are often dismissed as early-stage growing

pains, but our measurements reveal a deeper truth: *the very mechanisms that make IPFS decentralized—content addressing, DHTs, and Merkle-DAGs—are also its greatest performance liabilities.*

Although some prior studies [6, 19, 56, 58] have investigated the file retrieval performance of IPFS, showing high variance and generally inferior retrieval performance compared to traditional web protocols such as HTTP, they failed to diagnose the protocol-level root cause and uncover major bottleneck of client-side performance, let alone provide any feasible solutions to improve it. An in-depth understanding of the underlying content retrieval performance bottleneck is crucial, as improvements in this perspective can highly impact users' Internet experiences and potentially speed up the wide adoption of Web 3.0.

To fill this gap, we set to conduct a comprehensive study of IPFS's performance on content retrieval, which involves content discovery and downloading when a client accesses IPFS. In our study, we focus on revealing the underlying performance bottlenecks of IPFS and providing/evaluating our enhancements wherever appropriate. Our study leads to the following major findings.

- Compared to HTTP, downloading via IPFS shows significantly poorer performance, where on average HTTP has a throughput 8× over that of IPFS for large files. Our analysis traces this inefficiency to IPFS's block exchange protocol (Bitswap [3]), which enforces *sequential* block retrieval.
- Contrary to previous P2P norms [4], increasing replication (data sources) in IPFS fails to improve throughput due to its sequential download pattern caused by Bitswap, while inflating duplicate traffic by up to 82% in overhead. Further, the improvement of BBR (Bottleneck Bandwidth and Round-trip propagation time) over CUBIC is marginal in lossy environments because of the block request pattern of IPFS. The client requests blocks batch by batch, leading to low utilization of bandwidth.
- Although DHT is the default mechanism for content discovery in IPFS, in practice, Bitswap replaces its role as the majority (>80%) of the objects can be successfully discovered by Bitswap before the DHT search is launched. In addition, unresponsive peers could significantly delay the DHT lookup process, and in some cases, lead to failed lookups even for content that is otherwise accessible. Our investigation shows that carefully setting this threshold (*e.g.,* changing it from the current 1 second by default to 200 ms) could help improve the content lookup performance by better utilizing the DHT.

Those findings suggest that Bitswap contributes significantly in both the content discovery phase (*i.e.,* lookup) and file download phase (*i.e.,* retrieval) and is correlated with the issues we have identified. Therefore, to improve the file download performance of IPFS, we propose *Bitswap2.0*, which introduces two major enhancements to the default Bitswap protocol. First, to optimize Bitswap message exchanges (*e.g.,* number of messages and latency), *Bitswap2.0* leverages the local IPFS cache to preemptively retrieve the metadata that describes how a file is organized into a Merkle-DAG. Second, *Bitswap2.0* carefully balances the trade-off between the extra delay incurred when selecting the best peer to download from vs. the potential performance degradation if downloading from a randomly chosen peer.

We implement *Bitswap2.0* in a prototype and evaluate its performance. Our experimental results show that *Bitswap2.0* can eliminate duplicate data downloading and improve the download throughput of IPFS by 2× or more, especially for large files. *Bitswap2.0* is fully compatible with the current Bitswap. By no means *Bitswap2.0* is optimal. However, we hope that our findings and exploration will offer more insights regarding how to improve IPFS and bring its vision into a reality.

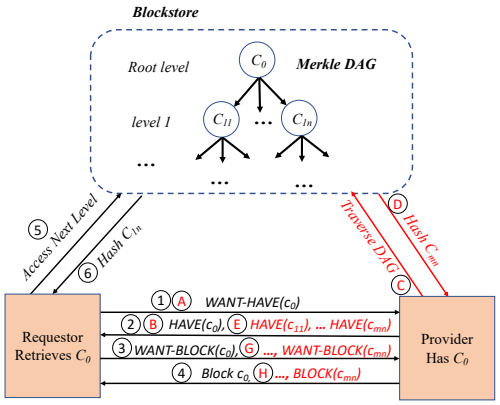


Figure 1: An overview of object retrieval in Bitswap (black) and Bitswap2.0 (red).

The remainder of the paper is as follows. A brief background of IPFS is presented in Section 2. We present our measurement and analysis in current IPFS regarding IPFS downloading in Section 3, and regarding lookup in Section 4. Section 5 presents our proposed design of *Bitswap2.0* and evaluation, respectively. We discuss the related work in Section 6, outline limitations and future work in Section 7, make concluding remarks in Section 8.

2 Background

Without relying on centralized servers, IPFS enables any user (referred to as a peer) to serve files or file chunks over a P2P network. Each peer is uniquely identified by a Peer Identifier (PeerID), and files/chunks are accessed via a Content Identifier (CID), ensuring data integrity through content-addressable storage. Files/chunks can be located using the Distributed Hash Table (DHT). Additionally, gateway services are provided to allow users outside the IPFS network to access IPFS content directly via HTTP. In this section, we briefly introduce key IPFS operations after an overview of the IPFS file organization.

2.1 IPFS File Structure

In IPFS, a file is stored as data blocks, each of which can accommodate up to 256 KB of data by default. Those blocks are organized in a data structure known as Merkle-DAG [5], essentially a tree with two types of blocks: raw block and reference block. When a file is published into IPFS, it is firstly split into one or multiple raw blocks, containing the original data. A raw block serves as a leaf node within the Merkle-DAG. Following the creation of raw blocks, reference blocks are generated on top of them to link those raw blocks together. Reference blocks themselves do not contain any raw data but instead hold metadata necessary for organizing the original file.

2.2 File Retrieval & Bitswap

The file retrieval in IPFS consists of two phases: lookup and downloading.[1] The lookup, also denoted as content discovery, locates the provider who has the root block of the target file while the downloading refers to transfering all the blocks once the provider of the root block is determined. The lookup phase can leverage two protocols: DHT and Bitswap. By design, DHT is the primary protocol for content lookup. However, a client will only launch DHT to look up the provider after the failure of a quick search via its neighbors within a certain time threshold. That is, before launching the DHT, the client asks all its neighbors for the target content using Bitswap messages. Such a time threshold is defined as ***provSearchDelay*** in the source code of IPFS and can be configured by the user. For downloading, it is primarily done via Bitswap and we shall discuss it later.

Lookup by DHT. The DHT lookup consists of three steps. First, the client performs a ***DHT walk*** to look up the provider record using the CID of the content. Initially, the client adds k closest peers to the CID in its routing table to the query queue and recursively queries the top α closest peers in parallel ($k = 20$, $\alpha = 10$ by default). The actual data transfer does not start until a provider record is found. Second, after getting the PeerID of the provider from the provider record, the client queries the DHT once again and retrieves the provider's physical address similar to the first step. Third, with the physical address, the client can connect to the provider. Once the lookup phase is complete, the client uses Bitswap to transfer the actual content.

File Retrieval by Bitswap. Figure 1 illustrates how an object (a file chunk, a file or a folder) is retrieved via Bitswap (in black font). The retrieval process always starts with looking up the CID of the root block c_0 and then retrieving the subsequent blocks. The retrieval process in current Bitswap works as follows: ①*: The requestor broadcasts a* WANT-HAVE *message to its connected peers for CID* c_0. ②*: Providers (those who have the root-CID block* c_0*) respond with a* HAVE *message.* ③④*: Upon receiving the* HAVE *message, the client sends a* WANT-BLOCK *message to the provider and then the provider responds with the requested block.* ⑤⑥*: The client extracts the content information to obtain the CID of the child blocks. The subsequent blocks are then retrieved by recursively traversing the Merkle-DAG, where a* WANT-BLOCK *message is directly sent to the provider of the root-CID block* c_0.

It is important to note several aspects of the file retrieval process in IPFS. First, if the requesting client does not receive any response for a certain CID within the configured time duration, ***provSearchDelay***, in step ② or ③, the requesting peer will query the DHT for the CID of the file to find a provider. Second, no lookup occurs for subsequent child blocks, and a *WANT-BLOCK* is directly sent to previously identified providers. If no block is returned, the client resorts to the DHT to look up the provider of the child block and fetch the block through the new provider. Thus, in this paper, the definition of lookup refers to the lookup of the root block only, which is different from prior studies [62], where they do not differentiate the lookup of child blocks and the root block. Third, in practice, block requests are issued in sequential batches (default with a maximum of 10 blocks per batch): the client treats each batch as a barrier, waiting for all requests in the current batch to complete (success or cancellation) before dispatching the next batch, which essentially means that the requests are not pipelined across batches. We refer to this behavior as sequential batch downloading in the remainder of the paper. Additional background information about Bitswap message types can be found in Appendix B.

[1] In this study, "retrieval" refers to the complete process required to fetch a file, including both lookup and downloading.

3 Downloading Measurement and Analysis

In this section, we begin with a measurement study conducted in both controlled environments and in the wild, focusing on the download performance of IPFS. Following the measurement results, we delve into the underlying causes of the observed deficiencies in IPFS downloading. To improve the performance, we propose and experimentally demonstrate the effectiveness of a new mechanism in its block exchange protocol, Bitswap.

3.1 Performance Measurement

Why HTTP as a baseline. In IPFS, Bitswap is the primary block exchange protocol for downloading, while a vast majority of today's web content is accessible via HTTP[2]. HTTP also remains the most widely deployed and standardized protocol for file delivery on the Internet [65], making it the natural benchmark for evaluating decentralized alternatives. Both protocols operate at the application layer and can significantly impact user experience depending on their download performance. By benchmarking IPFS against HTTP, we can clearly reveal IPFS's download bottlenecks relative to the dominant Web protocol, providing insights into what improvements are needed for IPFS to achieve competitive performance.

IPFS Performance in Controlled Environments. To evaluate the download performance of IPFS, we conducted a controlled experiment comparing its data retrieval efficiency against HTTP. All tests were performed using private AWS EC2 instances. We deployed senders on a t2.medium EC2 instance (2 vCPUs, 4 GB RAM) located on the U.S. West Coast for both HTTP and IPFS. The receivers were deployed on an identical instance on the U.S. East Coast. For test files, we generated dummy files of varying sizes (from 10 KB to 4 GB), with randomized data to ensure uniqueness and prevent caching effects.

For the **HTTP setup**, we used Nginx version 1.25.1 configured for HTTP/1.1 over TCP with range request support enabled. We controlled the client-side parallelism by issuing multiple concurrent range requests (up to 10, depending on file size) to a single HTTP server. The client downloaded each file in 256 KB segments by partitioning it into consecutive byte ranges and requesting those ranges in parallel via separate connections. This setup allows us to explicitly vary client-side parallelism while matching IPFS's download parallelism (up to 10 concurrent CID requests per batch). Caching was disabled by using Cache-Control: no-store. We selected HTTP/1.1 rather than HTTP/2 to avoid multiplexing and prioritization effects, which are not present in IPFS's Bitswap. HTTP/1.1 allows us to control parallelism explicitly through separate connections and processes. *Unless otherwise specified, the HTTP configuration remains consistent across all subsequent experiments.*

[2] We do not compare IPFS with HTTPS, as current Bitswap does not provide any encryption mechanism.

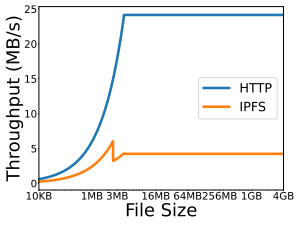


Figure 2: Downloading throughput comparison: IPFS vs. HTTP (*x-axis* in log scale).

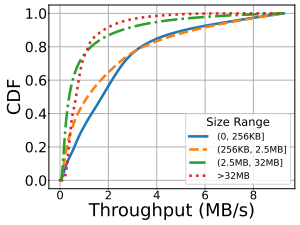


Figure 3: Throughput distribution of IPFS downloading across file size intervals in the wild.

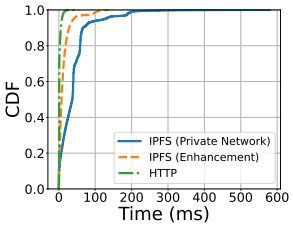


Figure 4: Inter-block arrival time (IBAT) distribution of both IPFS and HTTP for Figure 2.

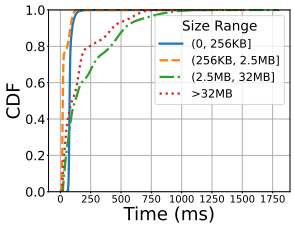


Figure 5: Distribution of IBAT across different size intervals (experiments in Figure 3).

For the **IPFS setup**, we ensured the client and provider were manually and consistently connected in the swarm, bypassing the lookup delays. Data retrieval used the default Bitswap behavior with 256 KB chunking. Like HTTP, IPFS communication was conducted over TCP at the transport layer. Note that HTTP operates under fundamentally different assumptions (*e.g.,* centralized, trusted servers and a largely non-adversarial setting), so we use it only as a Web2 baseline to expose bottlenecks rather than as a target design for IPFS.

For fairness, we ensure that (1) the reported downloading throughput excludes the Time-To-First-Byte (TTFB) to isolate steady-state download performance from connection setup or content discovery latency. As such, the throughput is calculated as the total file size divided by the time elapsed from the first byte received to the end of the transfer (*i.e.,* end-to-end latency minus TTFB), (2) the requested data was immediately discarded to eliminate the effects of local storage I/O on the results.

Figure 2 shows the throughput of IPFS and HTTP when downloading files of the same size in the controlled environments. Figure 2 covers a wide range of file sizes, spanning single-block objects ($\leq$ 256 KB) to very large files (> 1 GB) to capture common workloads and stress large transfers where batching effects dominate. We have two observations from this figure. (1) There is a sudden throughput drop at around 3 MB file size for IPFS download, while the HTTP downloading throughput keeps increasing with the file size until the throughput becomes steady. (2) The average throughput of HTTP is nearly 8× higher than that of IPFS, suggesting a much higher end-to-end download latency for IPFS.

IPFS Performance in the Wild. Considering that IPFS is a P2P network, its download performance is easily impacted by the network dynamics and different provider setups in practice. Thus, we further measure its download performance in the wild. Specifically, we randomly sampled 40K IPFS CIDs from a public gateway trace [19] and conducted the measurement on a t2.medium instance connected to the IPFS network. Figure 3 shows the download throughput of the 40K CIDs measured in the wild and the results are categorized into four size intervals. We observe that the average download throughput of files in (0,256 KB], (256 KB, 2.5 MB], (2.5 MB, 32 MB], and (32 MB, ∞) are 2.23 MB/s, 2.01 MB/s, 0.94 MB/s, and 0.89 MB/s, respectively. This indicates a clear download performance degradation of IPFS when the file size increases, particularly for files larger than 2.5 MB with a throughput of 2.02 MB/s at the 90^{th} percentile, compared to 5.47 MB/s for small files ($\leq$ 2.5 MB).

3.2 Bitswap Bottleneck Analysis

Bitswap operates at the block level, requesting blocks in batches (10 by default). As such, to understand IPFS's poorer throughput performance, we further analyzed the inter-block arrival time (IBAT) in the network trace during HTTP and IPFS downloading. Note that the term "block" here refers to the chunks of data being transferred. Figure 4 shows the IBAT distribution for IPFS and HTTP for the download operations presented in Figure 2. For HTTP, 80.31% of the IBAT is below 4 ms, indicating that the client can continuously get blocks when downloading via HTTP. For IPFS, the IBAT has a much longer tail with the 90^{th} value of 67.20 ms, compared to 5.65 ms of HTTP. That is, during IPFS downloading, roughly 10% of the blocks waited for more than 67 ms before the arrival of the next block, stalling the entire downloading process. We further present the IBAT distribution of the experiments depicted in Figure 3. As shown in Figure 5, the average IBAT of small files ($\leq$ 2.5MB) is much smaller than the large files, where the average IBAT of small file is 44.55 ms compared to 191.68 ms of large files. The longer IBAT of large files leads to their poorer performance. In addition, downloading in real-world conditions may experience even longer tails, where the IBAT can be up to 1,750 ms across all file sizes.

To understand why IPFS blocks experience a much longer inter-arrival time, we further analyze the Bitswap operations, and aim to answer the following questions.

(1) **Why does IPFS experience a longer IBAT compared to HTTP?** Bitswap requests blocks in batches. A batch ends only if all blocks in the batch are fetched. As such, faster blocks must wait for slower ones and the overall transfer time of a batch depends on the slowest RTT in the network. On the other hand, the batch-by-batch approach extends the interval between the last block of one batch and the first block of the next. In contrast, HTTP transfers each block independently and continuously, so the transfer time of one block is minimally affected by others. Furthermore, with Bitswap, when a client downloads from IPFS, it needs to acquire the blocks level by level whereas HTTP downloading uses sequential structure. As a result, the blocks located at the lower level of the Merkle tree have to wait until the upper-level blocks are downloaded and the corresponding links are obtained. The performance impact can be amplified if the downloading of upper-level blocks lags, blocking the overall downloading progress of the entire file. The delay caused by IPFS's batch-by-batch approach and Merkle tree file structure is evident in Figure 4, where a notable shift occurs at the 90^{th} percentile in the blue curve.

(2) **Why does there exist a sudden throughput drop as the file size exceeds 2.5 MB?** Small files (≤ 2.5 MB) typically consist of one or two layers and fewer than 10 blocks, while large files (> 2.5 MB) often have a more complex structure. This difference in complexity is reflected in the performance drop observed in Figure 2 and Figure 3. The downloading flow of the small files can be considered as a stream because the data blocks are requested in a single batch. However, for large files, their data has to be requested batch by batch, resulting in a higher IBAT and lower throughput.

(3) **Why does the IBAT of IPFS downloading experience long tail?** In addition to the extended waiting time for slow blocks and between batches, the initial provider might not have all the blocks of the requested objects. This forces the client to resort to the entire IPFS network, further prolonging the downloading. This is evident from Figure 5, where the tail of IBAT is much longer than that of the controlled experiment in Figure 4 as the downloading in the real-world situations is more unpredictable.

Another factor contributing to the slow downloading of IPFS is that the client requests each file individually through Bitswap. This leads to inefficiency when downloading a directory containing many small files. For instance, an NFT JSON collection might consist of thousands of small JSON files that occupy only a few megabytes in total or even less. Figure 5 further demonstrates this inefficiency: although downloading a single large block incurs no IBAT, the process of downloading a directory (which comprises many small files in the (0, 256 KB] range) results in a higher IBAT than downloading files in the (256 KB, 2.5 MB] range. Ideally, such downloads should complete within seconds. However, in IPFS, this process can take minutes, as the collection is divided into a large number of chunks, with each chunk representing a small file.

> **Takeaway 1:** The operational behavior of Bitswap is characterized by its batch-by-batch and file-by-file request pattern, along with a predominantly top-down approach to downloading data blocks. This design inherently affects the efficiency of bandwidth utilization and subsequently the overall throughput of data downloads.

3.3 Replication and Traffic

Location-based P2P protocols like BitTorrent benefit from data replication as they allow paralleled requests for different portions of the target content from various peers [32]. In contrast, IPFS, which uses a content-based approach, requires the location information of each chunk to assemble the entire target content. This requirement leads IPFS to adopt a message-based, sequential requesting pattern. Here, the target content is typically requested in a consecutive fashion, from beginning to end. This contrasts with BitTorrent's more flexible approach, which allows downloading from any part of the content regardless of the sequence order. This fundamental difference in design motivates our investigation into IPFS's performance under two critical scenarios: (1) multi-source downloads (retrieving content from multiple peers) and (2) network degradation (operating under varying packet loss rates and congestion control algorithms).

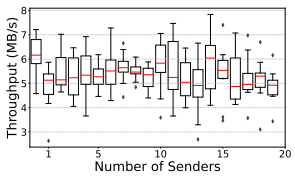


Figure 6: Downloading throughput as a function of replication level.

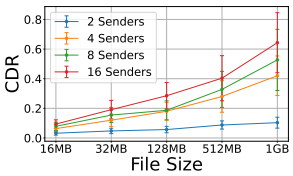


Figure 7: CDR as a function of the file sizes and the number of senders.

IPFS Download Under Multi-Source Scenarios. We conducted controlled experiments to evaluate IPFS's performance in multi-source environments. A single AWS EC2 instance (t2.medium, 2 vCPUs, 4 GB memory) served as the receiver, while multiple EC2 instances acted as senders, each holding a full copy of a 1 GB target file. These nodes formed a small autonomous system, with the receiver requesting the file while varying the number of senders (replications). Each configuration was tested for 1,000 iterations. Figure 6 illustrates the relationship between the number of senders (*x-axis*) and download throughput. Contrary to expectations, increasing the number of senders does not improve throughput—and in some cases, shows negligible impact. This aligns with IPFS's sequential batch retrieval mechanism: even with multiple sources, blocks are requested in sequential batches, forcing the requestor to wait iteratively for each batch to complete. Consequently, the parallelism advantages typical of P2P systems like BitTorrent are largely absent in IPFS. Beyond limited throughput gains, multi-source downloads in IPFS introduce significant duplicate traffic. Figure 7 quantifies this overhead using the *Content Duplication Ratio (CDR)*, defined as:

$$CDR = \frac{\text{Received data traffic} - \text{Actual file size}}{\text{Actual file size}}$$

From this figure, we observe that the total traffic can be up to 1.82×, indicating an 82% traffic overhead. It clearly shows that the CDR increases with both the number of senders and the file size. By examining the network trace, we find that the extra data traffic is due to the fact that some data blocks are downloaded multiple times. The reason is that, to optimize the download performance, an IPFS client may send multiple *WANT-BLOCK* messages when requesting a single block, resulting in downloading the same block from several peers (senders). However, this may not lead to positive impact on downloading performance as shown in Figure 6 because the downloading still has to be conducted sequentially. We also notice that the CDR increases with both file size and the number of seeds. To understand why CDR increases with file size, we looked into the Bitswap messages when requesting files. We found that larger files have a better chance of selecting more senders when requesting a block. This could be attributed to the complexity of the Merkle tree (i.e., the DAG) associated with larger files. Whenever the requestor requests a new block, it broadcasts a *WANT-LIST* message containing the information of all the blocks it wants to its connected peers. Thus, a large file that contains many blocks is more likely to identify more providers and receive more duplicate blocks. Similarly, we can observe that the CDR increases as the file size increases in the experiments. In addition, the duplicate traffic could

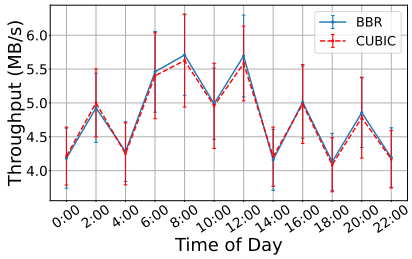


Figure 8: Throughput of IPFS downloading with BRR and CUBIC at different periods of a day.

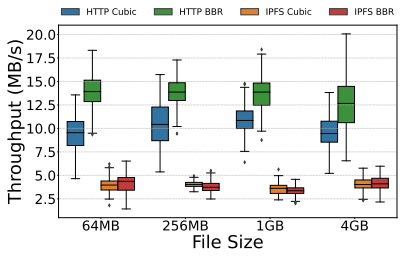


Figure 9: Throughput comparison of BBR and CUBIC under high latency environment with IPFS and HTTP.

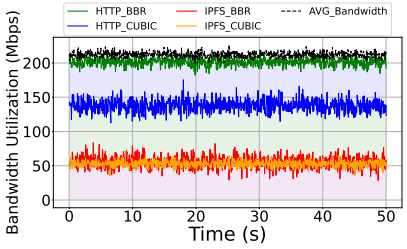


Figure 10: Network bandwidth usage under different downloading congestion control algorithms.

not account for performance acceleration as shown in Figure 6. Under IPFS's sequential batch downloading, the completion time of each batch is determined by its slowest block and thus duplicate traffic does not increase effective throughput.

> **Takeaway 2:** IPFS's sequential batch request mechanism limits its ability to leverage multi-source retrieval, which is a key advantage in traditional P2P systems Furthermore, this approach can introduce significant duplicate traffic, exacerbating inefficiencies in data transfer.

3.4 Congestion Control Impact Analysis

IPFS, as currently the largest P2P network in practice, its download performance could be largely impacted by the network dynamics even though its application layer protocol can make a big difference as we discussed in previous sections. That is, different congestion control algorithms underneath may also play a role in this process. For example, recently BBR is emergent [46]. In order to evaluate the impact of different congestion control algorithms, we modify the congestion control algorithm on the sender to use BBR or CUBIC. We select these two congestion control algorithms because CUBIC is the default option for Linux and macOS clients while BBR is widely employed by web applications these days.

Measurement Setup. In this experiment, we evaluate the impact of congestion control algorithms under two different settings, aiming to simulate low latency and high latency scenarios, respectively, and use throughput as the major evaluation metric. We use AWS instances (t2.xlarge, 4 vCPUs, 16 GB memory) as our clients and servers. To simulate the low latency scenario, we deploy both the client and the server in the same AWS region. We let the client retrieve a 128 MB dummy file published by the server 100 times and repeat the retrieval every two hours in six different AWS regions. The congestion control algorithm of the server is set to BBR or CUBIC. To simulate the high latency scenario, we deploy the client in the U.S East Coast and the server in the Pacific Southeast. We perform similar retrievals of 128 MB, 256 MB, 1 GB, and 4 GB files using HTTP (BBR or CUBIC) and IPFS (BBR or CUBIC). Each experiment is repeated 1,000 times.

Results and Analysis. Figure 8 presents the IPFS downloading throughput under BBR and CUBIC at different time periods of a day, which is conducted in the U.S East Coast. While the results show some variations over time, overall, we find that the performance difference due to different congestion control algorithms is not substantial in this low latency scenario. For example, the average throughput with BBR is 4.90 MB/s compared to 4.92 MB/s with CUBIC, and the difference is not statistically significant at the 99% confidence level. This result is not surprising as BBR is not meant to accelerate the downloading throughput in a decent environment.

Figure 9 shows the throughput comparison of HTTP and IPFS using BBR and CUBIC congestion control algorithms, performed in an environment with a packet loss rate of 2%. As shown by Figure 9, BBR is able to accelerate the downloading throughput of HTTP while it exhibits limited performance improvement for IPFS. The average throughput of HTTP BBR shows a 46.74% rise while IPFS BBR only sees a 0.62% rise across all file sizes. This limited improvement comes down to the fact that Bitswap requests blocks by blocks and it enters the next round only if the previous blocks are all received. As such, the upper bound of the transfer is actually the worst RTT of the packets sent, which is still the propagation delay of the route. Such behavior of Bitswap inevitably limits the usage of bandwidth. To further verify this assumption, we let the client with different downloading approaches fetch a very large file and observe the usage of the total bandwidth starting from the downloading stage through a long period. Figure 10 shows the bandwidth usage of HTTP_BBR, HTTP_CUBIC, IPFS_BBR and IPFS_CUBIC. As can be seen in Figure 10, IPFS enjoys very limited improvement in the high-latency environment. As a comparison, HTTP_BBR tries to make the best use of the available bandwidth and parallels the packet transfer as much as possible, thus accelerating the downloading.

> **Takeaway 3:** IPFS is application-limited rather than transport-limited. Bitswap's sequential batch downloading pattern introduces inter-batch gaps and underutilizes bandwidth, so changing the congestion control algorithm only has marginal gains in high latency environment. In contrast, HTTP keeps the pipeline full and therefore benefits from BBR, confirming that the bottleneck for IPFS lies above the transport layer.

4 DHT and Lookup Analysis

The last section shows that IPFS downloading is not comparable to HTTP downloading in terms of client-side download performance. As IPFS downloading involves content lookup first before the file

retrieval, we further look into the *lookup* phase in this section, specifically, the time taken to identify the provider.

For content lookup, IPFS has two alternatives, *DHT lookup* and *Bitswap* (Section 2). Upon initiating a file download, the client broadcasts the requests to all its neighbors, and the neighbors forward these messages to the network. If the provider is not resolved within a default 1-second duration (designated as *provSearchDelay* in the source code), a DHT lookup will be launched. This combined approach motivates us to explore the trade-off between these two alternatives. As such, we conduct a comparative measurement study on the lookup performance of Bitswap versus DHT. We also consider different types of content, popular NFT files and less frequently accessed test files, since NFT files are more likely to be quickly located by Bitswap. Furthermore, we investigate whether adjusting the *provSearchDelay* threshold could enhance lookup performance.

4.1 Lookup Comparison: DHT vs. Bitswap

Measurement Methodology. To compare these two lookup approaches, we first define the lookup stage for each. In Bitswap, we define the lookup stage for a chunk CID as the duration between initiating the retrieval and receiving a *HAVE* message from a provider. It is worth noting that the client might also directly receive the block without receiving the *HAVE* message if the requested block is too small. As for DHT lookup, we consider the lookup time as the duration between initiating the retrieval and resolving the peer record. This time interval involves two main steps: (1) identifying the PeerID of the content provider and (2) acquiring the provider's multiaddress, each requiring a DHT walk.

To measure the performance of DHT lookup, we configure the node in *dhtclient* mode and disable Bitswap's broadcast control, ensuring that lookups rely solely on the DHT. On the other hand, to disable DHT lookup, we set *provSearchDelay* to 60 seconds while measuring the performance of Bitswap lookup. As a result, if the target CID cannot be resolved within 60 seconds, we consider it inaccessible. If the provider of the target CID is resolved within 1 second, which is the default *provSearchDelay* threshold, we consider it a successful Bitswap lookup. In the experiment, we set up the IPFS client on t2.medium AWS instances in six regions and subsequently let the IPFS client fetch CID chunks by DHT lookup and Bitswap lookup, respectively. The CIDs are randomly sampled from a real-world IPFS network trace, which is passively collected by IPFS clients [9], in order to avoid potential sampling bias. Note that this trace is not a uniform sample of the global IPFS corpus, but reflects client-side demand (*i.e.,* what clients actually request), thus not systematically biased towards either popular or unpopular content. The network trace was collected in August, 2024. Each client retrieved 10,000 single-file chunks. For chunks that were unavailable, we excluded them from the results and moved to the next CID until 10,000 file chunks were fetched.

Measurement Results and Analysis. Figure 11 plots the CDF of the performance of Bitswap and DHT in the U.S East Coast . It includes the total duration of the retrieval (labeled as Bitswap and DHT) and the lookup (labeled as Bitswap_lookup and DHT_lookup) of both approaches. From the lookup performance comparison of DHT and Bitswap, we have several observations. (1) The success rate of Bitswap is 82.11% across all retrievals, indicating that most providers can be resolved within 1 second. (2) The performance of the Bitswap lookup outperforms the DHT lookup. In all retrievals, the lookup time of Bitswap takes 237.63 ms, 368.81 ms, and 402.90 ms at the 50^{th}, 75^{th}, and 99^{th} percentile, while the lookup time of DHT takes 427.58 ms, 1707.32 ms, 46089.97 ms at the 50^{th}, 75^{th}, and 99^{th} percentile.

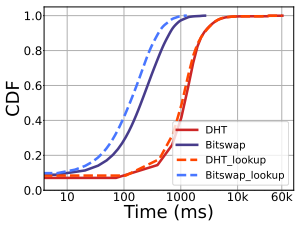


Figure 11: The CDF of total duration and lookup time of DHT and Bitswap lookup.

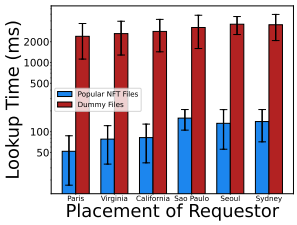


Figure 12: The lookup time comparison: popular NFT files vs. dummy files.

Lookup Performance Comparison: Popular vs. Unpopular Files. The lookup cost of an object is directly related to where it is stored. In the network trace [9] used above, a large portion of the CIDs originate from nodes hosted in cloud datacenters [58], which typically store popular content (*e.g.,* NFTs) and offer better connectivity. It remains unknown how the performance of lookup on unpopular files is. As such, we further seek to understand the performance differences of popular and unpopular (dummy) files. Note that we do not explicitly quantify popularity. Instead, we treat the top-ranked NFT files as *popular files*, since they are typically stored in cloud datacenters [10, 58] with reliable connectivity.

We first constructed an NFT dataset by web-scraping OpenSea [50] in late April 2023. Specifically, we enumerated the top 1,000 NFT collections and extracted token URLs that contained IPFS CIDs (either ipfs:// {CID} or gateway URLs embedding a CID), de-duplicating at the CID level. From this pool, we randomly sampled 10,000 CIDs spanning both NFT metadata objects and original media (*e.g.,* images, videos). Note that this sampling procedure is designed to reflect popular IPFS content, but it may introduce sampling bias by under-representing unpopular or rarely accessed objects.

We deployed AWS EC2 clients (t2.medium) in six regions and attempted to retrieve the sampled NFTs. A total of 10,370 CIDs were accessed, which exhibits high availability of those CIDs. We further let the clients request a single dummy file from the provider (t2.medium, located in central EU). We repeat the requests until 10,000 CIDs were successfully resolved. By the end of the experiment, a total of 12,202 CIDs have been accessed. We log the lookup time in both experiments.

Figure 12 presents the lookup time of NFTs conducted in all six regions, as well as for dummy files within the same region. As can be seen in Figure 12, there exists a huge performance gap between the lookup time of dummy files and popular files: the average lookup time for dummy files is 3,608 ms, while for popular files, it is 116 ms. In addition to this difference, the lookup time of these files also exhibits a large variation. We also observed significant differences in protocol behavior. The retrieval of popular NFT files achieved a Bitswap success rate of 98.56% while all dummy file lookups fell back to the DHT. This discrepancy is attributed to the fact that NFT

Table 1: Content availability across different lookup methods and content types.

Lookup Type / Content	Availability
Bitswap-Only Lookup (Figure 11)	25.16%
DHT-Only Lookup (Figure 11)	30.49%
Unpopular Dummy Files (Figure 12)	78.40%
Popular NFT Files (Figure 12)	99.55%

providers, typically hosted in cloud datacenters, maintain stable and well-connected peers, whereas the provider of the dummy file lacks a reliable P2P connection and often fails to maintain steady links with remote requestors.

Takeaway 4: In the real-world IPFS network trace, over 80% objects are discovered by Bitswap instead of DHT, and Bitswap outperforms DHT in terms of lookup performance. However, when it comes to unpopular files, IPFS is more likely to rely on its original DHT for content discovery, indicating the opportunistic nature of Bitswap lookup.

4.2 DHT Performance

In Section 4.1, we show that the DHT lookup performs significantly worse than Bitswap, with a long tail of latency that can extend beyond 30 seconds. Moreover, in our analysis of unpopular dummy files, we observe that the DHT sometimes fails to resolve the content, even when we verify that the content is present in the IPFS network. The content availability results in Section 4.1 are summarized in Table 1. These observations motivate us to investigate the causes of DHT underperformance and the issue of inadvertent unavailability introduced by relying on the DHT for content discovery.

Measurement Methodology. To investigate the root causes of DHT underperformance, we deploy two IPFS clients (t2.xlarge AWS instances) in the central EU. Each client is configured with a public IP address and allowed to join the network for an extended period to build a stable and populated routing table before initiating lookups. For the content, each client performs DHT lookups toward CIDs sampled from the previously used NFT traces and dummy files, as described in Section 4.1. We issue lookups for a total of 100, 000 CIDs (50,000 NFT CIDs and 50,000 dummy file CIDs) with a timeout threshold of 60 seconds per request. To ensure that each lookup performs a full DHT traversal, we disable local DHT cache checks and rely exclusively on the DHT for content discovery. As a result, every lookup in this experiment triggers a complete DHT walk. During the experiments, we log and calculate the following metrics.

- *Time-To-First-Response (TTFR)*: the delay between DHT walk and the receipt of the first peer response to a peer query.
- *Query Response Time (QRT)*: The duration between sending a query to a specific peer and receiving a corresponding response.
- *Key Resolution Time (KRT)*: The time taken to resolve the value associated with a key (*e.g.,* CID or PeerID) through a complete DHT walk.

Table 2: Summary of IPFS operations.

Operation	Successful	Unsuccessful
CID Accesses	89,883	10,117
Peer Queries	3,290,248	1,200,533
DHT Walks	123,710	32,309

- *Count of Hops*: The total number of iterations where the DHT query queue is updated with closer peers during the lookup process.

Results and Analysis. Table 2 summarizes the number of successful and unsuccessful DHT operations, showing an 89.88% availability rate. Figure 13 presents the CDF of TTFR, QRT, and KRT. Figure 14 presents the CDF of hop counts, comparing accessible versus inaccessible CIDs. As can be seen in this figure, about 90% of the DHT walks received their first response within 200 ms. However, the 90^{th} percentile of the query response time is about 2,358 ms, and the 90^{th} percentile of the key resolution time extends to 7,503 ms. That indicates, while one or two responsive peers often provide a fast initial response, the rest of the queried peers may be significantly slower or even unresponsive. For accessible CIDs, 95% of DHT walks complete within 20 hops. In contrast, 95% of DHT walks for inaccessible CIDs stall before reaching 30 hops. This suggests that inaccessible lookups often suffer from longer paths, in which case, the accumulation of unresponsive peers could dominate in the query queue and hinders the progress to the target key.

These findings highlight the need to examine how stragglers, the slow or unresponsive peers, affect both the performance and reliability of DHT lookups. Figure 15 further demonstrates this impact by presenting the *maximum stalled time ratio*, defined as the longest continuous waiting period (either for a response or during a query timeout) as a percentage of the total DHT walk duration. For incomplete lookups, we assume a 60-second DHT walk time. As shown in the figure, over 50% of successful DHT walks have a maximum stalled time that accounts for more than 50% of the total duration. The situation is significantly worse for inaccessible CIDs: in 80% of such cases, the maximum stalled time covers over 80% of the DHT walk time. This is primarily due to outbound queries being occupied by unresponsive peers early in the DHT walk. Once the concurrent query slots are blocked by such stragglers, the DHT walk is severely stalled, which could cost nearly 60 seconds, wasting time until the system timeout is reached.

Takeaway 5: While most DHT walks receive an initial response quickly, the presence of straggling or unresponsive peers could significantly delay the overall resolution. More critically, the lookup process grinds to a halt when outbound queries become congested by unresponsive peers, preventing further progress and ultimately resulting in failed lookups.

4.3 Dynamic DHT Launching Threshold

Measurement Methodology. The original purpose of Bitswap is to avoid querying DHT for every single block, thereby accelerating file retrieval and saving network resources. Determining the threshold for launching DHT is crucial for optimizing the lookup time of

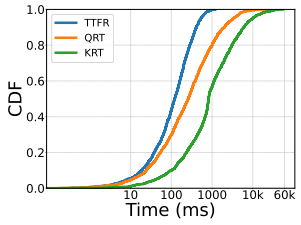


Figure 13: The CDF of TTFR, QRT and KRT.

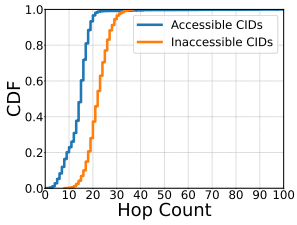


Figure 14: The CDF of hops among retrievals.

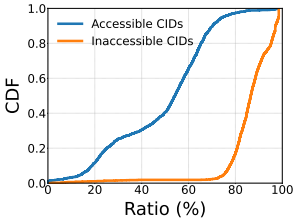


Figure 15: The CDF of the maximum stalled time ratio.

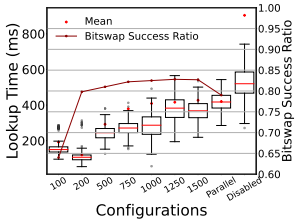


Figure 16: The lookup time at different *provsearchdelay*.

the root block. As we have demonstrated, Bitswap not only outperforms DHT in terms of lookup performance but also has a higher success rate. As such, we examine the impact of *provSearchDelay*[3] under various configurations. We repeated the experiment of Section 4.1 on the same t2.medium EC2 instance in the U.S East Coast, where the CIDs were sampled from a 2023 network trace [9]. In this experiment, we ensure that no CID is fetched more than once.

Results and Analysis. Figure 16 shows the result of the lookup time of a single root block under various configurations as well as its Bitswap success ratio. The default 1,000 ms threshold leads to an average lookup time of 409.53 ms compared to a lookup time of 121.87 ms with a threshold of 200 ms. This indicates that raising the threshold from 200 ms to 1,000 ms yields only a marginal increase in the Bitswap success ratio (79.83% to 82.80%), while delaying the DHT trigger and incurring longer lookup times. This improvement is due to the fact that a lower threshold significantly reduces the 90^{th} percentile of the lookup time. Unpopular files, which are less likely to be stored on neighbor peers, can be identified more quickly by triggering *provSearchDelay* threshold earlier. Specifically, the 90^{th} percentile under the 200 ms threshold is 795.37 ms compared to 1,988.39 ms under the 1,000 ms threshold. In the meantime, the Bitswap success ratio raises from 79.83% to 82.80%, only a minimal gain. When completely disabling the Bitswap lookup and resorting to DHT, the average lookup time becomes 534.73 ms. It is worth noting that the average lookup time is higher than that of Section 4.1. The reason is that, with fewer blocks stored on the neighbors, the Bitswap success rate is lower, making the lookup overhead higher. In addition, parallelizing the DHT lookup and Bitswap lookup does not improve the overall lookup performance given the average lookup time under such a scenario is 412.75 ms.

There have been discussions [1, 2] about whether to disable this threshold, change it to another value, or make it dynamic. We believe that this value is worth being kept but should be carefully set (e.g., a lower value such as 200 ms or even 100 ms). We argue that (1) Bitswap's opportunistic fashion is beneficial to the user experience as current IPFS shows centralized characteristics, where most files are stored on a centralized server and those centralized servers are easier to appear on the neighbor list [10, 58]. Thus, accessing some popular files may significantly benefit from such Bitswap broadcast messages. (2) Parallelizing DHT and Bitswap requests leads to inefficiencies in lookup. This is because both DHT and Bitswap are highly parallelized components, and parallelizing them further requires additional resources. This becomes problematic as the current IPFS framework limits the maximum number of processes that can be launched.

> **Takeaway 6:** As Bitswap exhibits a better performance against DHT in terms of lookup, an opportunistic Bitswap broadcast is worth the wait before launching the DHT. However, the lookup performance can be further improved by changing the threshold to a lower value (e.g., 200 ms). This adjustment allows for more effective utilization of the DHT, optimizing the balance between immediate broadcast response and comprehensive network search.

5 Design and Evaluation of *Bitswap2.0*

5.1 *Bitswap2.0* Design

Building on our earlier analysis, we have shown that IPFS file retrieval is bottlenecked by its own properties, Bitswap and DHT, for which a more substantial improvement is required. Thus, in this subsection, we present our design of *Bitswap2.0*, incorporating some new mechanisms to improve the IPFS downloading performance.

Problems that *Bitswap2.0* Aims to Address. In previous sections, we discussed how file blocks are transferred via Bitswap and its impact. Essentially, with Bitswap, during a file download process, **P(1)** the low-level blocks have to wait until high-level ones are fetched as a file is organized hierarchically using a Merkle tree structure, **P(2)** blocks must be requested sequentially, one file and then one batch of blocks at a time. Additionally, our DHT performance shows that the lookup performance is sensitive to the presence of stragglers and unresponsive peers, and **P(3)** outbound queries can become congested by the stragglers, pausing progress on provider discovery's critical path.

***Bitswap2.0* Overview.** Motivated by our analysis and findings, we propose *Bitswap2.0*, as sketched in Figure 1 (the part in red color). At a high level, *Bitswap2.0* incorporates the following enhancements. First, the provider peer piggybacks a compact manifest of descendant CIDs (the file's Merkle-DAG information) when it responds to the initial *WANT-HAVE* query of the root CID from the requesting client. This early Merkle DAG resolution lets the client learn the structure up front and avoid recursive parent-first traversal, addressing **P(1)**. Second, we relax the batch-by-batch constraint by enabling bounded concurrent requests without inter-batch barriers, so the client does not need to download strictly batch by batch, addressing **P(2)**. Finally, we propose a latency-aware peer selection algorithm that prioritizes responsive providers while balancing fairness. This reduces the likelihood of encountering unresponsive

[3]The default *provSearchDelay* threshold is 1 second in the version we used.

peers, thus mitigating **P(3)** and also accelerates end-to-end download performance. Next, we present the design details of *Bitswap2.0*.

Enhancement 1: Early Merkle Tree Resolution. The current Bitswap uses an inefficient Merkle tree traversal protocol, where it may take 2 RTTs to fetch a block and a block cannot be fetched until its parent block is downloaded. This block-by-block Merkle tree recursive traversing mechanism substantially impacts the transfer speed of IPFS. Hence, in *Bitswap2.0*, as soon as the query for the file's root-CID is received (i.e., the first block to download when a peer retrieves a file using IPFS), the provider (server) tries to obtain the information of all descendent CIDs in the root-CID's Merkle tree by checking its local cache, which is called the *blockstore*. The blockstore of a peer stores the information about the content that the peer has published or downloaded (Appendix B). In the current Bitswap, the requesting client has to obtain the parent block and extract its content information first before it can send the *WANT-BLOCK* messages for its child blocks. This works for all descendants of the tree in a recursive fashion.

In contrast, in *Bitswap2.0*, the seed peer directly replies with the Merkle tree information piggybacked in its reply (i.e., the *HAVE* message). This enhancement enables the client to immediately get access to the whole Merkle-tree structure without the need for traversing the entire file block structure one-by-one recursively. As a result, the client can fetch subsequent descendent CIDs in 1 RTT (whereas current Bitswap may take 2 RTTs). In the meantime, with the early Merkle DAG knowledge, the client can request blocks in a streamlined manner, eliminating inter-batch pauses. By contrast, in the original IPFS, enabling batch parallelization would require early DAG knowledge, so it defaults to sequential batches. Specifically, once the client learns the descendant CIDs up front, it builds a ready queue of blocks and immediately issues up to K concurrent *WANT-BLOCK* requests (default $K = 10$, same as current IPFS) across the DAG, refilling a slot as soon as any request completes or is canceled. Unlike prior studies [36] that employ a *know* message solely to boost Bitswap's success rate, our design optimizes the block exchange process and minimizes the data transfer latency.

Enhancement 2: Peer Selection Algorithm. *Bitswap2.0* introduces a new peer selection algorithm to improve the download throughput and minimizes redundant traffic. Recall in Section 2 we show that the current Bitswap: (1) enforces the client to send a *WANT-BLOCK* message upon receiving a *HAVE* message, and (2) may have the client resend the *WANT-BLOCK* message upon receiving more *HAVE* messages. As long as the client receives the block, it will send a *CANCEL* message to abort other transferring. While this redundancy transfer mechanism might improve the client-perceived latency by helping the client obtain the block from the fastest-responding peer, it inevitably leads to duplicate data traffic, a trade-off that may affect a client's resource usage efficiency.

Ideally, a client should select exactly one peer to request the block from, where the peer has the best transfer performance and is not overloaded. This requires a new peer selection algorithm as it introduces a new *trade-off*. That is, the peer that replies a *HAVE* message to the client the earliest is not necessarily the best candidate, since it might not have the highest bandwidth at that particular moment. The client needs to wait for more *HAVE* messages in order to pick the best candidate. But this leads to a higher delay. Thus, the new peer selection algorithm must carefully balance this trade-off.

To exploit this trade-off, we propose a new peer selection algorithm as follows. First, we take the peer latency and historically sent blocks into account when selecting peers. *Bitswap2.0* tracks the number of data blocks sent by a peer along the process and measures the latency to connected peers periodically. This makes *Bitswap2.0* latency-aware as it aims to find closer peers that can potentially improve downloading speed. *Bitswap2.0* uses the number of historically sent blocks to avoid overloading a particular peer, thereby achieving fairness. To do so, *Bitswap2.0* assigns two normalized scores $score_{latency}$ and $score_{\#blk}$ to each peer candidate, and calculates a moving average of each peer based on the following definition: $score_m = \alpha * cur\_score_m + (1-\alpha) * prev\_score_m$, and $cur\_score_m = \gamma * score_{latency} + (1-\gamma) * score_{\#blk}$, where the coefficients α and γ can be set empirically.

To select the best peer, *Bitswap2.0* utilizes a time window concept, whose duration can be dynamically adjusted. The time window is introduced so that the client can make more informed decisions when selecting peers. For each CID query, upon receiving the first *HAVE* message, the client starts a time window and only considers peers whose *HAVE* messages are received within the time window as potential peer candidates (the normalized $score_{latency}$ and $score_{\#blk}$ are based on the max and min values seen in the time window). Once the time window elapses, *Bitswap2.0* selects the peer that has the lowest score. This latency-aware peer selection algorithm is also applied during DHT provider discovery.

During a DHT query, candidates whose measured RTT exceeds a threshold s are discarded. Among the remaining peers, we order the candidates by XOR distance to the target key (Kademlia metric), using latency only as an admission filter. We do not include any block-load term at this stage, since DHT discovery is to locate the providers rather than transfer data.

Put It All Together. Figure 1 sketches the file retrieval steps under *Bitswap2.0* as follows (the enhancements of *Bitswap2.0* are marked in red in the figure): Ⓐ*: The client broadcasts* WANT-HAVE *messages to connected peers for root CID block* c_0. Ⓑ*: The server responds with a* HAVE *after querying* c_0. ⒸⒹ*: The server keeps on traversing the DAG and collects the CIDs of the descendants.* Ⓔ*: The server replies* HAVE *messages with descendant CIDs piggybacked.* Ⓕ*: The client selects the best peer among the providers who manage to send* HAVE *message in the time window.* ⒼⒽ*: The client sends the* WANT-BLOCK *messages after receiving the* HAVE *messages, then the server returns the requested blocks.*

5.2 *Bitswap2.0* Evaluation

To conduct the evaluation of *Bitswap2.0*, we deployed t2.xlarge EC2 instances with 4 vCPUs and 16 GB of memory running in different geographical locations. The clients are either configured with the current Bitswap or our enhancement *Bitswap2.0*. For the experiments, we again used randomly generated dummy files of varying sizes. Our evaluation is focused on the following metrics: (1) the downloading performance improvement, (2) the effectiveness of the peer selection algorithm, and (3) the overhead in terms of traffic.

Downloading Throughput. We conduct experiments to evaluate the throughput of *Bitswap2.0*. The IPFS nodes are deployed on identical EC2 instances in the real IPFS network. Similar to Section 3.1, for IPFS downloading, we deployed one sender on the West

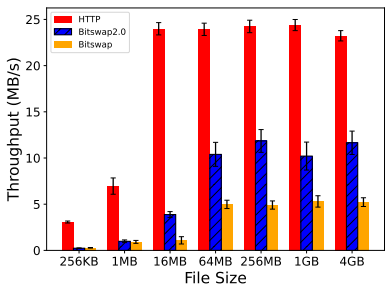


Figure 17: Throughput of different approaches as a function of file sizes.

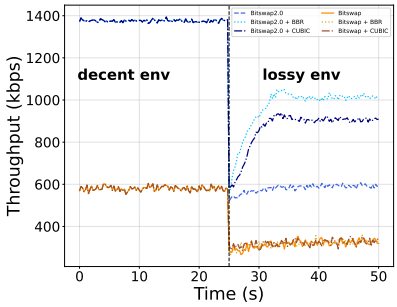


Figure 18: Comparison of Bitswap2.0 and Bitswap under lossy environment.

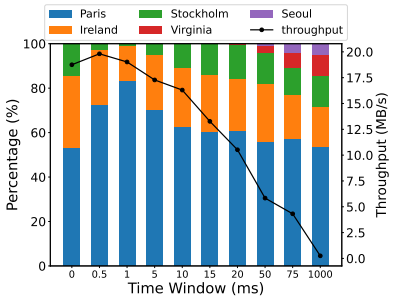


Figure 19: Breakdown of retrieval sources as a function of time window sizes.

Coast and one requestor on the East Coast. We let the requestor download the file by using *ipfs get* command. For each file, the downloading was repeated 100 times. HTTP downloading used the same configuration as in Section 3.1.

Figure 17 shows the throughput results with a 99% confidence interval. *Bitswap2.0* achieves similar performance as Bitswap for small files (e.g., the 256 KB and 1 MB files). For files that are 16 MB and larger, the throughput of *Bitswap2.0* is much better than Bitswap. For example, for 1 GB files, the throughput of *Bitswap2.0* is 2.23× of Bitswap. This is because, for large files, *Bitswap2.0* can significantly reduce the IBAT as shown in Figure 4, where the average IBAT of *Bitswap2.0* is 14.32 ms compared to 39.64 ms of the original IPFS. However, for small files, the lookup time (i.e., the time to identify the root block) is dominant in the entire process, and our enhancement in *Bitswap2.0* has little impact on when to send the HAVE message for the root block. In addition, the Merkle tree of small files is relatively shallow, typically with one or two levels. Thus, the benefit of our enhancement to Bitswap for retrieving information of the subsequent child blocks diminishes.

One the other hand, we observe that the performance gap between HTTP and *Bitswap2.0* becomes smaller when the downloaded file sizes increase. The average throughput of *Bitswap2.0* is 7.03 MB/s compared to 18.55 MB/s of HTTP transfer for large files of 256 MB, 1 GB, and 4 GB. This is because, compared to HTTP, *Bitswap2.0* still incurs some extra cost, for example, the blockstore lookups and the delay to select a peer.

Figure 18 further shows that *Bitswap2.0* can exploit congestion control to use more bandwidth under lossy conditions (packet loss rate = 2%). As shown in this figure, when the network becomes congested at 25 s, both *Bitswap2.0* and original Bitswap flow drop, but *Bitswap2.0* responds strongly to the congestion control algorithms: its lossy interval-average throughput increases from 581 kbps (baseline) to 965 kbps with BBR (+66.0%) and 866 kbps with CUBIC (+48.9%). In contrast, Bitswap's average changes little from 316 kbps (baseline) to 340 kbps with BBR (+7.59%) and 321 kbps with CUBIC (+1.47%). This result stems from Bitswap's sequential, batch-by-batch request pattern, which introduces pauses between rounds, underutilizing link capacity and leaving little room for congestion control algorithms to accelerate. In other words, the bottleneck is at the application layer, not the transport. As a result, faster congestion control yields only marginal gains for the original Bitswap.

Effectiveness of Peer Selection Algorithm. Next, we evaluate if our peer selection algorithm can identify the best peer and does not cause a large delay. We deployed one client in Frankfurt. Then we deployed five seed peers in each of the following locations, Paris, Ireland, Stockholm, Virginia, and Seoul. The client aimed to download a file that is available at all other peers. During the download, we logged the data block origin (i.e., where a data block came from) and the throughput. We repeated the download test 100 times for different time window sizes and plotted the distribution of the block origins of a 1GB file in Figure 19.

Each bar shows the percentage of blocks that each region accounts for. For example, when the time window is set to 0 (i.e., the client immediately responds with a *WANT-BLOCK* upon receiving a *HAVE* message), 53% of the data blocks were downloaded from Paris, 32% from Ireland, and 15% from Stockholm. Note that the best peer is supposed to be from Paris in terms of latency as *Bitswap2.0* makes the peer selection decision as Paris has the lowest RTT (around 10 ms compared to 15 ms of Ireland and 25 ms of Stockholm). This means that 53% of *Bitswap2.0*'s peer selection choices were optimal and the corresponding throughput was 18.75 MB/s. When the time window was increased to 0.5 ms, 72% of the peer selection is set to Paris and the corresponding throughput increased slightly to 19.81 MB/s, the highest among all time windows. As a time window size of 0 in *Bitswap2.0* reflects the same behavior of the current Bitswap (except that the requesting peer in *Bitswap2.0* does not keep sending *WANT-BLOCK* messages), these results indicate the peer selection in *Bitswap2.0* is more effective in selecting the best peer to download from.

As we further increased the time window to 20 ms, the client eventually managed to receive blocks from Virginia. When the time window becomes 1 second (the biggest time window we tested), we can see that the distribution was more balanced among all five locations than other smaller time windows. This is because the client was able to receive almost all *HAVE* messages for the CID from all five locations. A downside of this, however, is significantly dropped throughput to only 0.38MB/s. This indicates that an appropriate time window size can help the client make better decisions and improve the throughput. We further compare the lookup performance of *Bitswap2.0* and Bitswap under different *provSearchDelay* settings in Figure 20 (Appendix C), showing that *Bitswap2.0* reduces the lookup time by an average of 35.60%.

Table 3: Bitswap traffic and message breakdown at the client end (the current Bitswap).

File Size	Traffic Total	Traffic In	Traffic Out	# Msg In	# Msg Out	Msg Size	Dup. Traffic
256 KB	286.902 KB	286.830 KB	73.505 B	7	88	0.630 B	30.830 KB
1 MB	1.317 MB	1.317 MB	528.941 B	35	281	1.395 B	324.295 KB
16 MB	22.492 MB	22.339 MB	156.673 KB	592	5,311	22.688 B	6.337 MB
64 MB	90.031 MB	89.384 MB	662.575 KB	2,307	17,936	29.867 B	25.380 MB
256 MB	402.712 MB	391.616 MB	11.096 MB	7,876	74,804	0.123 KB	135.505 MB
1 GB	2.033 GB	1.816 GB	222.211 MB	32,768	315,397	0.471 KB	0.810 GB
4 GB	9.254 GB	7.009 GB	2.245 GB	155,939	1,368,919	1.283 KB	2.989 GB

Table 4: Bitswap traffic and message breakdown at the client end (*Bitswap2.0*).

File Size	Traffic Total	Traffic In	Traffic Out	# Msg In	# Msg Out	Msg Size	Dup. Traffic
256 KB	256.028 KB	256.028 KB	0.596 B	9	1	1.710 B	0
1 MB	1.001 MB	1.001 MB	5.672 B	39	5	3.621 B	0
16 MB	16.014 MB	16.014 MB	196.308 B	563	79	14.854 B	0
64 MB	64.219 MB	64.215 MB	4.369 KB	2,256	273	59.392 B	0
256 MB	259.610 MB	259.504 MB	108.416 KB	8,916	1,063	247.158 B	0
1 GB	1.063 GB	1.061 GB	1.928 MB	32,512	5,004	0.850 KB	0
4 GB	4.625 GB	4.611 GB	14.199 MB	140,358	16,631	2.892 KB	0

Message Overhead. IPFS relies on Bitswap for control message exchanges. During the downloading process, the IPFS nodes receive and send a lot of Bitswap messages. To reduce the message exchange, *Bitswap2.0* selects one server to request a block. Furthermore, since *Bitswap2.0* no longer requires the exchange of *WANT-LIST* messages, this leads to a reduced number of response messages and reduced storage cost. But in *Bitswap2.0*, when a server node responds, it includes the link information of subsequent blocks, which incurs extra traffic. To evaluate the traffic overhead of both Bitswap and *Bitswap2.0*, we deployed both the IPFS provider and requestor on t2.xlarge EC2 instances located in the U.S. East Coast. The requestor retrieves files of varying sizes, ranging from 256KB to 4GB. Each file transfer is repeated 100 times. We capture the network traffic using Wireshark [70] and record both packet sizes and the total transferred payload.

Table 3 and Table 4 show the traffic breakdown during our experiments, which are averaged across the 100 runs. The total traffic includes the aggregate traffic of all received data blocks and the Bitswap messages. The incoming traffic includes the file blocks and *HAVE* messages, while the outgoing traffic includes *WANT* messages. *Bitswap2.0* incurs less traffic than Bitswap due to the reduced number of message exchanges and the complete elimination of duplicate block traffic. *Bitswap2.0* also requires significantly fewer messages to be exchanged compared to Bitswap. In most cases, the messages exchanged under *Bitswap2.0* are only 10%-15% of that under Bitswap. The cost that *Bitswap2.0* needs to pay is the larger message size. One should note that this cost is negligible and outweighed by the gains that *Bitswap2.0* introduces.

6 Related Work

IPFS Measurement. Measurement studies focusing on IPFS have garnered considerable attention due to the critical role of decentralized storage in Web 3.0 architectures. These studies have spanned several dimensions, including file writing/chunking performance [6, 56], DHT publicizing performance [62, 63, 67], content retrieval performance [6, 45, 56, 58, 62, 67], network topology analysis [9, 34], and analysis of centralization [10, 57, 58]. Prior studies towards content retrieval performance often overlook the impact of IPFS file structure and the block transfer protocol, Bitswap. Specifically, their content retrieval measurements either only consider single block or small files [62, 67], or focus primarily on end-to-end latency without a fine-grained analysis [6, 44, 56]. Although studies [58, 62] divide the file retrieval process into content discovery and content fetch, they did not study the impact of Bitswap. Our study expands upon those studies by providing a comprehensive evaluation of various performance metrics and root cause analysis of the performance bottlenecks, including content fetching and discovery, with new insights and improvement directions.

IPFS Performance Improvement. Recent efforts to improve IPFS performance fall into two main directions: incorporating dedicated peers [62, 67] and protocol redesign [23, 63]. Dedicated peers, such as gateways, network indexers or Hydra boosters [67], are centralized components that complement IPFS's decentralized architecture. They cache provider records or content to accelerate retrieval, which comes at the cost of decentralization. To accelerate content publication performance, predictive models have been used to accelerate DHT *PUT* operations by optimistically storing provider records on likely target peers [63].

IPFS Security and Privacy. A fully public, decentralized, and permissionless P2P network like IPFS, by design, lacks mechanisms for identity verification, leaving it susceptible to Sybil-based attacks [26]. Two representative Sybil-based threats in IPFS are peer eclipse attacks [52] and content eclipse attacks [17, 59]. The former manipulates a victim's routing table by flooding it with Sybil nodes, thereby isolating the victim from honest peers, while the latter targets content availability by surrounding the provider record with malicious nodes that drop or suppress responses. Moreover, the use of Bitswap messages raises additional privacy concerns, as they can potentially reveal individual usage patterns [9]. To address this issue, several countermeasures have been proposed, including trickle-spreading [21] , hash obfuscation [36] and privacy-preserving Bitswap protocol [22].

Decentralized File Systems. Although IPFS provides permissionless decentralized storage and retrieval, it lacks built-in data

persistence mechanisms. Other decentralized storage services generally employ two main redundancy strategies (full replication and erasure coding) to ensure data persistence. For example, both Filecoin [39] and Arweave [69] integrate a blockchain-based incentive layer with a native cryptocurrency (FIL and AR, respectively) to economically incentivize long-term storage. Filecoin builds directly on top of IPFS, requiring storage providers to maintain full replicas of uniquely encoded file copies (typically slow to retrieve), while Arweave further employs a proof-of-access consensus mechanism to retain more hot copies locally, thereby improving retrieval performance. In contrast, systems such as Sia [64], Storj [60], and Swarm [27] adopt erasure coding to achieve durability with reduced storage overhead compared to full replication strategy. Most recently, Walrus [20] employs a two-dimensional erasure coding scheme to achieve both low storage overhead and efficient self-healing capabilities under network partitions.

P2P Optimization. In the context of traditional P2P file sharing systems, lots of efforts have been made to mitigate the issues of free-riding and promote fairness, such as incentive mechanisms [33, 39, 54, 61]. On the other hand, P2P systems are also widely used for streaming services, requiring a more stringent demand of user experience. In order to enhance the quality of service, some prior studies have focused on modeling and theoretical analysis of piece selection algorithms [16, 37, 49, 73] and request-peer selection algorithms [43, 68]. Furthermore, several studies have explored improving the network efficiency by ISP traffic optimization [7, 8, 40].

Other P2P Network Evaluations. P2P networks have maintained an important role since the early Internet with numerous applications in addition to IPFS. The most well-known applications are Bitcoin [47] and BitTorrent [4]. On one hand, Bitcoin demonstrates the potential of P2P network in resisting censorship. Various statistics of Bitcoin have been analyzed, such as transaction characteristics [15, 31], propagation delay [24, 25], node geographic distribution [42, 51] and user behavior [13, 29]. On the other hand, BitTorrent presents the efficiency of P2P network in large-scale file distribution with measurement studies focusing on its traffic characteristics [32, 41], DHT crawling and analysis [35, 66, 71] and performance [12, 28].

7 Discussion

In this section, we discuss the limitations of our study and future improvements to IPFS.

Decentralization vs. Performance. From a performance perspective, the key advantage of a P2P system lies in its crowdsourcing nature: as more peers join, they collectively contribute bandwidth and resources. This capability is leveraged by classic P2P systems such as BitTorrent, which are primarily designed to share files and accelerate download speeds through multi-source downloading. In contrast, IPFS places greater emphasis on decentralization as its core design principle. This difference in objectives leads to a key architectural divergence: while BitTorrent often relies on trackers for rapid peer discovery in addition to DHT, IPFS largely depends on DHT for content resolution and operates at a finer block-level granularity through content-addressed blocks and its Merkle DAG structure. This content-based architecture endows IPFS with its decentralized nature, but also introduces higher coordination overhead and a more complex block exchange process. To this end, we have designed *Bitswap2.0*, which optimizes the block exchange process by piggybacking descendant CID information earlier and improving peer routing through better peer selection. However, there is still a performance gap between *Bitswap2.0* and HTTP as shown in Section 5.2. This is mainly due to the extra delay for IPFS nodes to look up the block information and resolve the file structure before the actual data transfer starts. Future research could focus on further integrating block exchange optimizations (*e.g.,* caching 'hot' blocks), transport layer improvements (*e.g.,* a systematic evaluation of QUIC in IPFS) and improving peer discovery mechanisms (*e.g.,* pre-dial nearest candidates), in order to narrow the gap with HTTP while preserving IPFS's decentralization guarantees.

***Bitswap2.0* Design Trade-offs.** Bitswap2.0's early Merkle-DAG piggybacking creates a computational amplification risk: a malicious client can repeatedly solicit large manifests and abort, forcing providers to enumerate DAGs and transmit metadata. This can be mitigated with per-peer token-bucket rate limiting or caps on manifest size/depth. The client may also serve manifests only after minimal proof of interest (e.g., successful root block transfer) and cache recent enumerations to avoid repeated traversal cost. Additionally, Bitswap messages pose privacy concerns as they expose information (*e.g.,* CID request history) to any peer that receives them [9]. *Bitswap2.0* focuses on enhancing the block exchange efficiency and does not directly address the privacy aspect. However, *Bitswap2.0* does avoid the leakage of descendant CIDs as they are probed via unicast, exchanged only between the requestor and provider. In contrast, in the original Bitswap, both root and descendant CIDs are commonly probed via broadcast. However, the dominant privacy leakage risk still arises from the initial broadcast of the root CID, which can allow observers to infer the file's entire Merkle-DAG structure.

8 Conclusion

IPFS stands as a pioneering effort to decentralize the web, yet our study reveals the hidden costs of its content-centric architecture. By dissecting the client-side performance, we expose critical tensions between Web 3.0's ideological goals and practical experience. Our empirical results and findings reveal performance deficiencies and bottlenecks in IPFS's content discovery and retrieval process. To address these issues, we have explored and demonstrated the potential of different improvements, ranging from modifications to system configurations to improve lookup performance, to the design and implementation of a new file transfer protocol, *Bitswap2.0*, that employs an enhanced message-transferring mechanism. By no means should those improvements be considered optimal for file transferring in IPFS. However, we hope that providing those solutions can offer insights into the future development of IPFS and Web 3.0.

Acknowledgment

We thank the shepherd and anonymous reviewers for their valuable feedback. This work was partially supported by the National Science Foundation under CNS-2322860. Some results presented in this paper were obtained using CloudBank [48], supported by the NSF award CNS-1925001.

References

[1] 2022. Discussion about provSearchDelay. https://github.com/ipfs/kubo/issues/8807. Accessed: 2024-04-01.

[2] 2022. A measurement study towards the effectiveness of IPFS Bitswap. https://github.com/probe-lab/network-measurements/blob/master/results/rfm16-bitswap-discovery-effectiveness.md. Accessed: 2024-04-03.

[3] 2025. Bitswap. https://docs.ipfs.tech/concepts/bitswap/. Accessed: 2024-04-30.

[4] 2025. BitTorrent. https://www.bittorrent.com/. Accessed: 2025-05-14.

[5] 2025. Merkle Directed Acyclic Graphs (DAGs). https://docs.ipfs.tech/concepts/merkle-dag/. Accessed: 2024-03-05.

[6] Omar Abdullah Lajam and Tarek Ahmed Helmy. 2021. Performance evaluation of ipfs in private networks. In *2021 4th International Conference on Data Storage and Data Engineering*. 77–84.

[7] Vinay Aggarwal, Obi Akonjang, and Anja Feldmann. 2008. Improving user and ISP experience through ISP-aided P2P locality. In *IEEE INFOCOM Workshops 2008*. IEEE, 1–6.

[8] Vinay Aggarwal, Anja Feldmann, and Christian Scheideler. 2007. Can ISPs and P2P users cooperate for improved performance? *ACM SIGCOMM Computer Communication Review* 37, 3 (2007), 29–40.

[9] Leonhard Balduf, Sebastian Henningsen, Martin Florian, Sebastian Rust, and Björn Scheuermann. 2022. Monitoring data requests in decentralized data storage systems: A case study of IPFS. In *2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 658–668.

[10] Leonhard Balduf, Maciej Korczyński, Onur Ascigil, Navin V Keizer, George Pavlou, Björn Scheuermann, and Michał Król. 2023. The cloud strikes back: Investigating the decentralization of ipfs. In *Proceedings of the 2023 ACM on Internet Measurement Conference*. 391–405.

[11] Juan Benet. 2014. Ipfs-content addressed, versioned, p2p file system. *arXiv preprint arXiv:1407.3561* (2014).

[12] Ashwin Bharambe, Cormac Herley, and Venkata N Padmanabhan. 2005. Understanding and deconstructing bittorrent performance. In *Proc. ACM Sigmetrics*.

[13] Alex Biryukov, Dmitry Khovratovich, and Ivan Pustogarov. 2014. Deanonymisation of clients in Bitcoin P2P network. In *Proceedings of the 2014 ACM SIGSAC conference on computer and communications security*. 15–29.

[14] Vitalik Buterin et al. 2014. A next-generation smart contract and decentralized application platform. *white paper* 3, 37 (2014), 2–1.

[15] Xucan Chen, Mohammad Al Hasan, Xintao Wu, Pavel Skums, Mohammad Javad Feizollahi, Marie Ouellet, Eric L Sevigny, David Maimon, and Yubao Wu. 2019. Characteristics of bitcoin transactions on cryptomarkets. In *Security, Privacy, and Anonymity in Computation, Communication, and Storage: 12th International Conference, SpaCCS 2019, Atlanta, GA, USA, July 14–17, 2019, Proceedings 12*. Springer, 261–276.

[16] Jeng-Long Chiang, Yin-Yeh Tseng, and Wen-Tsuen Chen. 2011. Interest-Intended Piece Selection in BitTorrent-like peer-to-peer file sharing systems. *Journal of parallel and distributed computing* 71, 6 (2011), 879–888.

[17] Thibault Cholez and Claudia-Lavinia Ignat. 2024. Sybil attack strikes again: denying content access in IPFS with a single computer. In *Proceedings of the 19th International Conference on Availability, Reliability and Security*. 1–7.

[18] Cisco Systems. 2023. Annual Internet Report (2018–2023) White Paper. https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html Accessed: October 2023.

[19] Pedro Ákos Costa, João Leitão, and Yannis Psaras. 2023. Studying the workload of a fully decentralized Web3 system: IPFS. In *IFIP International Conference on Distributed Applications and Interoperable Systems*. Springer, 20–36.

[20] George Danezis, Giacomo Giuliari, Eleftherios Kokoris Kogias, Markus Legner, Jean-Pierre Smith, Alberto Sonnino, and Karl Wüst. 2025. Walrus: An Efficient Decentralized Storage Network. *arXiv preprint arXiv:2505.05370* (2025).

[21] Erik Daniel, Marcel Ebert, and Florian Tschorsch. 2023. Improving Bitswap Privacy with Forwarding and Source Obfuscation. In *2023 IEEE 48th Conference on Local Computer Networks (LCN)*. 1–4. https://doi.org/10.1109/LCN58197.2023.10223322

[22] Erik Daniel and Florian Tschorsch. 2023. Privacy-Enhanced Content Discovery for Bitswap. In *2023 IFIP Networking Conference (IFIP Networking)*. 1–9. https://doi.org/10.23919/IFIPNetworking57963.2023.10186387

[23] Alfonso De la Rocha, David Dias, and Yiannis Psaras. 2021. Accelerating content routing with bitswap: A multi-path file transfer protocol in ipfs and filecoin. *San Francisco, CA, USA* (2021), 11.

[24] Christian Decker and Roger Wattenhofer. 2013. Information propagation in the bitcoin network. In *IEEE P2P 2013 Proceedings*. IEEE, 1–10.

[25] Joan Antoni Donet Donet, Cristina Pérez-Sola, and Jordi Herrera-Joancomartí. 2014. The bitcoin P2P network. In *International conference on financial cryptography and data security*. Springer, 87–102.

[26] Peter Druschel, Frans Kaashoek, and Antony Rowstron. 2003. *Peer-to-Peer Systems: First International Workshop, IPTPS 2002, Cambridge, MA, USA, March 7-8, 2002, Revised Papers*. Vol. 2429. Springer.

[27] Ethereum Foundation. 2025. Swarm: Decentralised Storage and Communication System. https://www.ethswarm.org/. Accessed: 2025-08-30.

[28] Bin Fan, John CS Lui, and Dah-Ming Chiu. 2008. The design trade-offs of BitTorrent-like file sharing protocols. *IEEE/ACM Transactions on Networking* 17, 2 (2008), 365–376.

[29] Giulia Fanti and Pramod Viswanath. 2017. Deanonymization in the bitcoin P2P network. *Advances in Neural Information Processing Systems* 30 (2017).

[30] R Fielding, M Nottingham, and J Reschke. 2022. RFC 9110: HTTP Semantics.

[31] Befekadu G Gebraselase, Bjarne E Helvik, and Yuming Jiang. 2021. Transaction characteristics of bitcoin. In *2021 IFIP/IEEE International Symposium on Integrated Network Management (IM)*. IEEE, 544–550.

[32] Lei Guo, Songqing Chen, Zhen Xiao, Enhua Tan, Xiaoning Ding, and Xiaodong Zhang. 2005. Measurements, analysis, and modeling of bittorrent-like systems. In *Proceedings of the 5th ACM SIGCOMM Conference on Internet Measurement*. 4–4.

[33] Songlin He, Yuan Lu, Qiang Tang, Guiling Wang, and Chase Qishi Wu. 2022. Blockchain-based p2p content delivery with monetary incentivization and fairness guarantee. *IEEE Transactions on Parallel and Distributed Systems* 34, 2 (2022), 746–765.

[34] Sebastian Henningsen, Martin Florian, Sebastian Rust, and Björn Scheuermann. 2020. Mapping the interplanetary filesystem. In *2020 IFIP Networking Conference (Networking)*. IEEE, 289–297.

[35] Konrad Junemann, Philipp Andelfinger, Jochen Dinger, and Hannes Hartenstein. 2010. Bitmon: A tool for automated monitoring of the bittorrent dht. In *2010 IEEE Tenth International Conference on Peer-to-Peer Computing (P2P)*. IEEE, 1–2.

[36] Christos Karapapas, George Xylomenos, and George C Polyzos. 2024. Enhancing IPFS Bitswap. In *International Conference on Information and Communication Technology for Intelligent Systems*. Springer, 189–199.

[37] Nouman Khan, Mehrdad Moharrami, and Vijay Subramanian. 2020. Stable and efficient piece-selection in multiple swarm bittorrent-like peer-to-peer networks. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 1153–1162.

[38] Yekta Kocaogullar, Eric Osterweil, and Lixia Zhang. 2024. Towards a Decentralized Internet Namespace. In *Proceedings of the ACM Conext-2024 Workshop on the Decentralization of the Internet*. 36–41.

[39] Protocol Labs. 2017. Filecoin: A Decentralized Storage Network. https://filecoin.io/filecoin.pdf. Accessed: 2025-08-30.

[40] Minghong Lin, John CS Lui, and Dah-Ming Chiu. 2009. An isp-friendly file distribution protocol: analysis, design, and implementation. *IEEE Transactions on Parallel and Distributed Systems* 21, 9 (2009), 1317–1329.

[41] Zhang Lin, Wang Hao, and Zhong Shibing. 2012. A measurement study on BitTorrent traffic behaviors over IPv6. In *2012 IEEE International Conference on Computer Science and Automation Engineering (CSAE)*, Vol. 3. IEEE, 354–357.

[42] Matthias Lischke and Benjamin Fabian. 2016. Analyzing the bitcoin network: The first four years. *Future internet* 8, 1 (2016), 7.

[43] Nianwang Liu, Zheng Wen, Kwan L Yeung, and Zhibin Lei. 2012. Request-peer selection for load-balancing in P2P live streaming systems. In *2012 IEEE Wireless Communications and Networking Conference (WCNC)*. IEEE, 3227–3232.

[44] Xiaoyue Ma, Lannan Luo, and Qiang Zeng. 2024. From one thousand pages of specification to unveiling hidden bugs: Large language model assisted fuzzing of matter {IoT} devices. In *33rd USENIX Security Symposium (USENIX Security 24)*. 4783–4800.

[45] Xiaoyue Ma, Qiang Zeng, Haotian Chi, and Lannan Luo. 2023. No more companion apps hacking but one dongle: Hub-based blackbox fuzzing of iot firmware. In *Proceedings of the 21st Annual International Conference on Mobile Systems, Applications and Services*. 205–218.

[46] Ayush Mishra, Wee Han Tiu, and Ben Leong. 2022. Are we heading towards a BBR-dominant Internet?. In *Proceedings of the 22nd ACM Internet Measurement Conference*. 538–550.

[47] Satoshi Nakamoto. 2008. Bitcoin whitepaper. *URL: https://bitcoin. org/bitcoin. pdf-(: 17.07. 2019)* (2008).

[48] Michael Norman, Vince Kellen, Shava Smallen, Brian DeMeulle, Shawn Strande, Ed Lazowska, Naomi Alterman, Rob Fatland, Sarah Stone, Amanda Tan, et al. 2021. Cloudbank: Managed services to simplify cloud access for computer science research and education. In *Practice and Experience in Advanced Research Computing*. 1–4.

[49] Ilkka Norros, Hannu Reittu, and Timo Eirola. 2011. On the stability of two-chunk file-sharing systems. *Queueing Systems* 67 (2011), 183–206.

[50] OpenSea. 2025. OpenSea, the Largest NFT Marketplace. https://opensea.io/. Accessed: 2025-05-01.

[51] Sehyun Park, Seongwon Im, Youhwan Seol, and Jeongyeup Paek. 2019. Nodes in the bitcoin network: Comparative measurement study and survey. *IEEE Access* 7 (2019), 57009–57022.

[52] Bernd Prünster, Alexander Marsalek, and Thomas Zefferer. 2022. Total Eclipse of the Heart - Disrupting the InterPlanetary File System. In *USENIX Security Symposium*.

[53] Yiannis Psaras and David Dias. 2020. The InterPlanetary File System and the Filecoin Network. In *2020 50th Annual IEEE-IFIP International Conference on Dependable Systems and Networks-Supplemental Volume (DSN-S)*. 80–80. https://doi.org/10.1109/DSN-S50200.2020.00043

[54] Rameez Rahman, Michel Meulpolder, David Hales, Johan Pouwelse, Dick Epema, and Henk Sips. 2010. Improving efficiency and fairness in p2p systems with effort-based incentives. In *2010 IEEE International Conference on Communications*. IEEE, 1–5.
[55] Partha Pratim Ray. 2023. Web3: A comprehensive review on background, technologies, applications, zero-trust architectures, challenges and future directions. *Internet of Things and Cyber-Physical Systems* (2023).
[56] Jiajie Shen, Yi Li, Yangfan Zhou, and Xin Wang. 2019. Understanding I/O Performance of IPFS Storage: A Client's Perspective. In *2019 IEEE/ACM 27th International Symposium on Quality of Service (IWQoS)*.
[57] Ruizhe Shi, Ruizhi Cheng, Yuqi Fu, Bo Han, Yue Cheng, and Songqing Chen. 2025. Centralization in the Decentralized Web: Challenges and Opportunities in IPFS Data Management. In *Proceedings of the ACM on Web Conference 2025*. 4068–4076.
[58] Ruizhe Shi, Ruizhi Cheng, Bo Han, Yue Cheng, and Songqing Chen. 2024. A Closer Look into IPFS: Accessibility, Content, and Performance. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 8, 2 (2024), 1–31.
[59] Srivatsan Sridhar, Onur Ascigil, Navin Keizer, François Genon, Sébastien Pierre, Yiannis Psaras, Etienne Rivière, and Michał Król. 2024. Content Censorship in the InterPlanetary File System. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS 2024)*. The Internet Society, 1–17. https://doi.org/10.14722/ndss.2024.23153
[60] Storj Labs, Inc. 2018. Storj: A Decentralized Cloud Storage Network Framework. https://github.com/storj/whitepaper. Accessed: 2025-08-30.
[61] Guang Tan and Stephen A Jarvis. 2008. A payment-based incentive and service differentiation scheme for peer-to-peer streaming broadcast. *IEEE transactions on parallel and distributed systems* 19, 7 (2008), 940–953.
[62] Dennis Trautwein, Aravindh Raman, Gareth Tyson, Ignacio Castro, Will Scott, Moritz Schubotz, Bela Gipp, and Yiannis Psaras. 2022. Design and evaluation of IPFS: a storage layer for the decentralized web. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 739–752.
[63] Dennis Trautwein, Yiluo Wei, Yiannis Psaras, Moritz Schubotz, Ignacio Castro, Bela Gipp, and Gareth Tyson. 2024. Ipfs in the fast lane: Accelerating record storage with optimistic provide. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 1920–1929.
[64] David Vorick and Luke Champine. 2014. Sia: Simple decentralized storage. *Retrieved May* 8 (2014), 2018.
[65] W3Techs. 2025. Usage Statistics of HTTP for Websites. https://w3techs.com/technologies/details/ce-http2 Accessed: September 16, 2025.
[66] Liang Wang and Jussi Kangasharju. 2013. Measuring large-scale distributed systems: case of bittorrent mainline dht. In *IEEE P2P 2013 Proceedings*. IEEE, 1–10.
[67] Yiluo Wei, Dennis Trautwein, Yiannis Psaras, Ignacio Castro, Will Scott, Aravindh Raman, and Gareth Tyson. 2024. The Eternal Tussle: Exploring the Role of Centralization in {IPFS}. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 441–454.
[68] Zheng Wen, Nianwang Liu, Kwan L Yeung, and Zhibin Lei. 2011. Closest playback-point first: A new peer selection algorithm for P2P VoD systems. In *2011 IEEE Global Telecommunications Conference-GLOBECOM 2011*. IEEE, 1–5.
[69] Sam Williams, Viktor Diordiiev, Lev Berman, and Ivan Uemlianin. 2019. Arweave: A protocol for economically sustainable information permanence. *Arweave Yellow Paper* 67 (2019).
[70] Wireshark Foundation. 2025. Wireshark — Go Deep. https://www.wireshark.org/. Accessed: 2025-05-15.
[71] Zhang Xinxing, Tian Zhihong, and Zhang Luchen. 2016. A measurement study on mainline dht and magnet link. In *2016 IEEE First International Conference on Data Science in Cyberspace (DSC)*. IEEE, 11–19.
[72] Lixia Zhang, Alexander Afanasyev, Jeffrey Burke, Van Jacobson, KC Claffy, Patrick Crowley, Christos Papadopoulos, Lan Wang, and Beichuan Zhang. 2014. Named data networking. *ACM SIGCOMM Computer Communication Review* 44, 3 (2014), 66–73.
[73] Yipeng Zhou, Dah Ming Chiu, and John CS Lui. 2007. A simple model for analyzing P2P streaming protocols. In *2007 IEEE International Conference on Network Protocols*. IEEE, 226–235.

A Ethics

Our measurement study is conducted under an IRB approval. In the study, we started with a publicly available dataset, which only contained anonymized data including CIDs and peerIDs. Our experiment did not involve any IP level analysis, nor did we try to track any interest of any user, as such analysis is not within our scope.

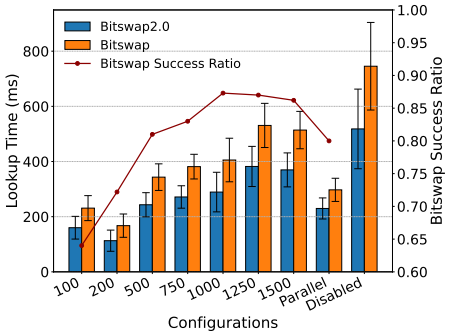


Figure 20: Lookup Time under different *provSearchDelay* settings

B IPFS Background

IPFS Block Service. IPFS is comprised of a number of layers and components that work together to provide its unique functionality. The block service is the backbone among these layers, responsible for local data storage and retrieval. The block service further contains two components: a local blockstore and Bitswap. The blockstore is a local cache for the data blocks. Bitswap is the core protocol for data transfer in IPFS. The blockstore provides various APIs for data management and seamlessly interacts with the Bitswap protocol to exchange a block. For example, when a peer is trying to retrieve a certain CID, the first thing is to look it up in the blockstore. If the block corresponding to the CID is locally stored, it immediately returns the block by *getBlock()* API. If it is not locally stored, it will notify Bitswap to fetch this block remotely.

Bitswap Messaging Types. The core of Bitswap's data transfer mechanism is by a series of messages exchanged between nodes. The following are five types of messages in the current Bitswap.

- *WANT-HAVE*. The client initiates the retrieval of a block by broadcasting *WANT-HAVE* messages to see if other peers have the certain block. It contains a list of CIDs that it wants, which is called *wantlist*.
- *WANT-BLOCK*. Namely, this message is used to directly request a block.
- *HAVE* and *DONT-HAVE*. These two message types are the responses to the *WANT-HAVE* or *WANT-BLOCK*.
- *CANCEL*. This type of message is to retract the previous requests of CIDs. When the client receives a block, it will broadcast *CANCEL* to notify the peers that this certain CID is no longer desired.

C Additional Measurement Results

***Bitswap2.0* vs. Bitswap under Different *provSearchDelay* settings.** Figure 20 presents the lookup performance under different *provSearchDelay* settings. Experiments were run on t2.xlarge instances on the U.S. East Coast using the 2023 network trace. For each configuration, we issued 10,000 CID lookups. From the figure we observe: (1) the Bitswap success ratio averages 79.86%, indicating that most lookups are still served by Bitswap; and (2) *Bitswap2.0* consistently outperforms Bitswap, primarily because *Bitswap2.0* accelerates DHT routing via peer selection. In the "Disabled" configuration (Bitswap lookup off), the average IPFS lookup time is 809 ms, whereas *Bitswap2.0* with peer selection reduces the average to 521 ms.